# Assignment: Approximating Natural Language

From Shannon to Modern AI

Basics of AI

**Assigned:** Wednesday, October 22, 2025
**Due:** Tuesday, November 4, 2025 at 11:59 PM

## 1 Background: Claude Shannon and the Birth of Information Theory

In 1948, Claude Shannon published "A Mathematical Theory of Communication," fundamentally changing how we understand information, communication, and language. Shannon, often called the "father of information theory," demonstrated that language could be modeled as a stochastic process—a sequence of probabilistic events.

Shannon's work showed that English text has statistical properties that can be captured at different levels of approximation, from random letter generation to increasingly sophisticated n-gram models. These insights laid the groundwork for modern natural language processing, machine translation, and generative AI systems. The Markov chains Shannon explored are direct ancestors of today's language models, including the transformer architectures powering systems like GPT and Claude.

In this assignment, you'll recreate Shannon's experiments using modern tools, building your own text generator that captures the statistical essence of different authors' writing styles.

**Important:** Before beginning implementation, read **Appendix A** to see Shannon's original text approximation examples. These demonstrate the exact progression from random gibberish to coherent English that your generators should replicate.

## 2 Learning Objectives

By completing this assignment, you will:

- Understand the statistical nature of natural language
- Implement n-gram language models from scratch
- Explore how increasing context (from unigrams to trigrams) improves text generation
- Gain hands-on experience with Markov chains and probability distributions
- Appreciate the historical foundations of modern NLP and generative AI

## 3 Part 1: Data Collection and Preparation (20 points)

### 3.1 Step 1.1: Text Files and Starter Code Provided

The following materials have been provided in the Brightspace module:

**Cleaned Literary Texts:**

- **Jane Austen**: `austen_pride_prejudice.txt`
- **Mark Twain**: `twain_tom_sawyer.txt`
- **Arthur Conan Doyle**: `doyle_sherlock_holmes.txt`

**Preprocessing Starter Code:**

- `starter_preprocess.py` - Complete text cleaning and tokenization utilities

*Note: These texts were downloaded from Project Gutenberg (`https://www.gutenberg.org`) and cleaned to remove headers and footers. Each contains the complete novel in UTF-8 encoding. The starter code handles all text preprocessing so you can focus on the statistical analysis and generation algorithms - the core of Shannon's insights!*

### 3.2 Step 1.2: Understanding the Preprocessing

The provided `starter_preprocess.py` includes two main classes:

- `TextPreprocessor` - Handles text cleaning, normalization, and tokenization
- `FrequencyAnalyzer` - Calculates n-gram frequencies and probabilities

Study this code and test it with the provided texts. You may use it as-is or modify it for your needs.

**Deliverable:** `analyze.py` that uses the preprocessing utilities to analyze all three authors

## 4 Part 2: Statistical Analysis (25 points)

### 4.1 Step 2.1: Character-Level Analysis

For each author, calculate:

- Single character frequencies (including space)
- Bigram frequencies (two-character sequences)
- Trigram frequencies (three-character sequences)

### 4.2 Step 2.2: Word-Level Analysis

For each author, calculate:

- Unigram word frequencies
- Bigram word frequencies
- Trigram word frequencies

### 4.3 Step 2.3: Sentence Structure Analysis

For each author, determine:

- Distribution of sentence lengths (in words)
- Average sentence length
- Standard deviation of sentence lengths
- Most common sentence patterns (optional: based on POS tags)

**Deliverable:** `analyze.py` that generates frequency tables and statistics, saved as JSON files using the provided `FrequencyAnalyzer` class

# 5 Part 3: Text Generation Engine (35 points)

## 5.1 Step 3.1: Core Generator Implementation

Create a `TextGenerator` class that can:

- Load frequency tables for a specified author
- Generate text using different approximation levels:
  - Zero-order (random characters with equal probability)
  - First-order character (characters with English frequencies)
  - Second-order character (bigram-based)
  - Third-order character (trigram-based)
  - First-order word (unigram words)
  - Second-order word (bigram words)
  - Third-order word (trigram words)

## 5.2 Step 3.2: Sentence-Aware Generation

Enhance your generator to:

- Generate sentences matching the author's length distribution
- Properly capitalize sentence starts
- Add appropriate punctuation

## 5.3 Step 3.3: Anchor Word Integration

Add functionality to include specific "anchor words" in generated text:

- Words must appear at least once in the generated output
- Should feel naturally integrated, not forced
- Tip: Generate multiple candidates and select ones containing anchor words

**Deliverable:** `generator.py` with the `TextGenerator` class

# 6 Part 4: Command-Line Interface (20 points)

Create a CLI tool `shannon_gen.py` with the following commands:

```
# Analyze texts and build frequency tables
python shannon_gen.py analyze --author austen --file pride_prejudice.txt
python shannon_gen.py analyze --author twain --file tom_sawyer.txt
python shannon_gen.py analyze --author doyle --file sherlock_holmes.txt

# Generate text with different approximation levels
python shannon_gen.py generate --author austen --level char-2 --length 100
python shannon_gen.py generate --author twain --level word-3 --sentences 5

# Generate with anchor words (must include these words at least once)
python shannon_gen.py generate --author doyle --level word-2 --sentences 3 \
    --anchors "elementary,Watson,deduce"

# Compare approximation levels for same author
```

```
python shannon_gen.py compare --author austen --sentences 2

# Blend styles (bonus)
python shannon_gen.py blend --authors "austen,twain" --level word-2 --
    sentences 3
```

Listing 1: Required CLI Commands

### 6.1 Required Command Options

- `--author`: Select author (austen, twain, doyle)
- `--level`: Approximation level (char-0 through word-3)
- `--length`: Number of characters to generate (for character-level)
- `--sentences`: Number of sentences to generate (for word-level)
- `--anchors`: Comma-separated words that must appear with natural frequency

**Deliverable:** `shannon_gen.py` with full CLI implementation

## 7 Part 5: Analysis and Reflection (20 points)

Write a 2–3 page report addressing:

1. **Quality Comparison**: How does the quality of generated text improve from zero-order to third-order approximations? Provide specific examples.
2. **Author Distinctiveness**: Can you identify which author is being mimicked at different approximation levels? At what level does author style become recognizable?
3. **Anchor Word Integration**: Discuss the challenge of incorporating specific words while maintaining natural flow. How did you solve this problem?
4. **Modern Connections**: How do these n-gram models relate to modern language models? What are the key limitations of n-gram approaches that modern neural networks address?
5. **Shannon's Insight**: Reflect on Shannon's key insight about language as a stochastic process. How does this experiment demonstrate that insight?

## 8 Submission Requirements

Submit a zip file containing:

- All Python code (`.py` files)
- Generated frequency tables (JSON files)
- Sample outputs from each approximation level for each author
- Your analysis report (PDF)
- A `README.md` with installation and usage instructions
- A `requirements.txt` file

# 9 Grading Rubric

| Component | Weight |
|---|---|
| Code Quality (clean, well-documented, modular) | 30% |
| Correctness (accurate frequencies and generation) | 30% |
| CLI Functionality (all commands work as specified) | 20% |
| Analysis Report (thoughtful reflection and examples) | 20% |
| **Total** | **100%** |

# 10 Resources

- Original Shannon paper: "A Mathematical Theory of Communication" (1948)
- Project Gutenberg: `https://www.gutenberg.org/`
- Python libraries to consider: `nltk`, `collections.Counter`, `numpy`, `argparse`
- N-gram smoothing techniques: Chen & Goodman (1999)

# 11 Example Outputs & Implementation Guidance

To help you understand what to expect and avoid common pitfalls, here are example outputs for each approximation level:

## 11.1 Character-Level Approximations

**Zero-Order (Random characters, equal probability)**

```
XKQMZ WPFGH JKLXV QWRTY BVCXZ ASDFG HJKLP OIUYT
```

*What you see:* Complete chaos. No structure whatsoever.

**First-Order Character (Character frequencies match English)**

```
OEET RNIS HAEA TLNE EOITR ASHN TEIO RAELS DNHT
```

*What you see:* More vowels (E, A, O) and common consonants (T, N, R). Looks slightly less alien but still gibberish.

**Second-Order Character (Bigram-based)**

```
THER ANBE INTH ATOES OURE ISTH ANDIN WAVES THEON
```

*What you see:* Common letter pairs emerge! "TH", "ER", "IN", "AN" – starting to see English-like fragments.

**Third-Order Character (Trigram-based)**

```
THE ANDING WHICOULD HAVERY THISE WITHAT SHOUGHT
```

*What you see:* Actual word fragments! "THE", "AND", "WHICH", "HAVE" – getting much closer to real English.

## 11.2 Word-Level Approximations (Jane Austen Example)

**First-Order Word (Words with correct frequencies)**

```
"the she very and elizabeth feelings to jane her good the was
elizabeth manner the of affection be most the marriage"
```

*What you see:* Real words! Common words like "the" appear often, but no grammatical sense.

**Second-Order Word (Bigram-based)**

```
"she was not be able to her father and mother was very much
to be so very agreeable and the whole party"
```

*What you see:* Phrases start making sense! "she was", "to her father", "very much" – grammatical patterns emerging.

**Third-Order Word (Trigram-based)**

```
"elizabeth could not help observing as she turned over some
books on the table that he was very much in love with her sister"
```

*What you see:* Nearly coherent sentences! Local context makes sense, though the overall meaning might still wander.

## 12 Academic Integrity

This is an individual assignment. While you may discuss concepts with classmates, all code and analysis must be your own. Cite any external resources or code snippets used.

*"The fundamental problem of communication is that of reproducing at one point either exactly or approximately a message selected at another point."*
— Claude Shannon, 1948

## Appendix A: The Series of Approximations to English

To give a visual idea of how this series of processes approaches a language, typical sequences in the approximations to English have been constructed and are given below. In all cases we have assumed a 27-symbol "alphabet," the 26 letters and a space.

**Zero-order approximation (symbols independent and equiprobable).**

XFOML RXKHRJFFJUJ ZLPWCFWKCYJ FFJEYVKCQSGHYD QPAAMKBZAACIBZLHJQD

**First-order approximation (symbols independent but with frequencies as in English).**

OCRO HLI RGWR NMIELWIS EU LL NBNESEBYA TH EEI ALHENHTTPA OOBTTVA NAH BRL

**Second-order approximation (digram structure as in English).**

ON IE ANTSOUTINYS ARE T INCTORE ST BE S DEAMY ACHIN D ILONASIVE TUCOOWE AT TEASONARE FUSO TIZIN ANDY TOBE SEACE CTISBE

**Third-order approximation (trigram structure as in English).**

IN NO IST LAT WHEY CRATICT FROURE BIRS GROCID PONDENOME OF DEMONSTURES OF THE REPTAGIN IS REGOACTIONA OF CRE

**4. First-order word approximation.** Rather than continue with tetragrams, etc., we turn to word units. Here words are chosen independently but with their appropriate frequencies.

REPRESENTING AND SPEEDILY IS AN GOOD APT OR COME CAN DIFFERENT NATURAL HERE HE THE A IN CAME THE TO OF TO EXPERT GRAY COME TO FURNISHES THE LINE MESSAGE HAD BE THESE

**5. Second-order word approximation.** The word transition probabilities are correct but no further structure is included.

THE HEAD AND IN FRONTAL ATTACK ON AN ENGLISH WRITER THAT THE CHARACTER OF THIS POINT IS THEREFORE ANOTHER METHOD FOR THE LETTERS THAT THE TIME OF WHO EVER TOLD THE PROBLEM FOR AN UNEXPECTED

The resemblance to ordinary English text increases quite noticeably at each of the above steps. Note that these samples are constructed and are not taken from any book. The trigram word construction, for example, was book at random pages from a book and the words were found. Here with the first word, continued the reading until the trigram was found and recorded the word following. This was continued until several pages of words and trigrams were filled. The procedure for the case of pairs of words was similar but simpler. It is felt in this way as the result of statistical structure more or less matching the original English material. The last case from the trigram construction, for example, appears to be a random selection of words and phrases from the original book but broken down and reconstructed. An examination of the original book however shows that these particular combinations of words never actually occur. Thus while the overall structure has been reconstructed, the local detailed structure has been disturbed.

To carry this analysis further one would construct trigrams, four-grams, etc. As $n$ increases the correspondence with ordinary English writing increases. For $n$ sufficiently large the generated material would be in a reasonable approximation to ordinary English text. It appears that a sufficiently complex machine environment requires considering very large portions of message with a minimum of computation. The approximation which increases requires appreciably but not explosively large statistics.

Turning to another type book model it would be somewhat later frequency characteristics of certain words or, for example, one could predict the frequency characteristics of pairs of words or phrases. From such analyses it follows that a text can be constructed and placed so sentences without grammatical structure can be placed so sentences while it randomly ordered produces reasonable English text approximately constructed from such statistics. One can again here approximate writing with three frequencies, then with four frequencies. In another type book it could

be determined the letter frequency characteristics, then the pair letter frequency characteristics, etc.

In the last case the resulting structure has some of the simplicity and in fact approximates quite closely what we often call "broken English."

### 12.1 Historical Context and Modern Relevance

Shannon's experiments in 1948 laid the groundwork for:

- Modern statistical language models
- N-gram based text generation systems
- The theoretical foundations for measuring text quality via information theory
- Contemporary neural language models (GPT, BERT, etc.)

Shannon noted that going to third-order word approximation would require an enormous amount of text for the transition table, presaging the data requirements of modern large language models.

---

*These examples demonstrate the core principle you'll implement: as the order of your Markov model increases, the generated text becomes increasingly coherent and realistic, but at the cost of increased computational complexity and data requirements.*